

BÁO CÁO KIỂM TOÁN MÃ NGUỒN TOÀN DIỆN BẢO MẬT & HIỆU NĂNG

*Phân tích chuyên sâu các lỗ hổng bảo mật, lỗi logic vận hành
và đề xuất giải pháp tái cấu trúc tối ưu hiệu năng tối đa*

Dự án: JDKTrap Project (Roblox Launcher Extension)

Ngày thực hiện: 12 tháng 06, 2026

Kiểm toán viên: Antigravity AI Coding Assistant (DeepMind)

I. TÓM TẮT DỰ ÁN

JDKTrap là một phiên bản launcher tùy chỉnh nâng cao dành cho Roblox, được phát triển nhằm cung cấp các công cụ tối ưu hóa đồ họa, tinh chỉnh Fast Flags hệ thống, tích hợp VPN (SwiftTunnel) và các tiện ích tối ưu RAM, CPU để nâng cao FPS khi chơi game.

Báo cáo kiểm toán này được thực hiện nhằm rà soát một cách nghiêm túc toàn bộ mã nguồn của dự án, tập trung tìm kiếm các lỗ hổng bảo mật (security vulnerabilities), các lỗi logic vận hành (logic bugs) và các điểm nghẽn hiệu suất (performance bottlenecks) để đề xuất giải pháp xử lý triệt để.

Kết quả đánh giá cho thấy dự án sở hữu bộ tính năng phong phú, tuy nhiên đang gặp phải một số lỗi nghiêm trọng:

- Lỗ hổng bảo mật:** Lưu trữ token OAuth bằng thuật toán XOR tĩnh dễ bị giải mã; nguy cơ Path Traversal trong cơ chế quản lý tệp tin backup; nguy cơ DLL Hijacking khi nạp thư viện động.
- Lỗi logic vận hành:** Hạ mức ưu tiên của game Roblox xuống Below Normal gây giật lag; lạm dụng API EmptyWorkingSet gây Page Fault liên tục; lỗi logic truy vấn module đồ họa DirectX 12.
- Điểm nghẽn hiệu suất:** Sử dụng GC.Collect() định kỳ 5 giây gây nghẽn ThreadPool; cấp phát liên tục tài nguyên D3D11 trong luồng lặp stress GPU gây phân mảnh VRAM.

Khuyến nghị tổng quát: Cần lập tức áp dụng DPAPI để mã hóa token, chuẩn hóa các đường dẫn và tái cấu trúc luồng tối ưu hóa GPU/RAM thành cơ chế khởi tạo một lần (Singleton/Cached).

II. LỖ HỔNG BẢO MẬT (SECURITY VULNERABILITIES)

1. Lưu trữ Token OAuth thiếu an toàn (High Severity) [Nghiêm Trọng - High]

Vị trí: *Integrations/SwiftTunnel/SwiftTunnelAuthManager.cs (Dòng: 221-229, 250-258)*

Chi tiết: Khi ghi nhớ trạng thái đăng nhập, tệp tin 'swifttunnel_session.dat' được tạo ra để lưu trữ JWT Access Token và Refresh Token của người dùng. Tuy nhiên, thay vì sử dụng cơ chế mã hóa an toàn, ứng dụng chỉ sử dụng một thuật toán XOR tính với khóa cứng 8-byte là 'SwiftTun' (0x53, 0x77, 0x69, 0x66, 0x74, 0x54, 0x75, 0x6E) kết hợp mã hóa Base64 để ghi file.

Nguy cơ: Do khóa mã hóa được ghi cứng (hardcoded) trong mã nguồn, bất kỳ phần mềm độc hại nào chạy trong quyền người dùng (User context) hoặc một kẻ tấn công có quyền đọc file đều dễ dàng giải mã và chiếm đoạt toàn bộ session, vượt qua hoàn toàn cơ chế bảo mật OAuth để truy cập trái phép API.

Giải pháp đề xuất: Thay thế thuật toán XOR bằng cơ chế mã hóa DPAPI của Windows thông qua thư viện ProtectedData:

```
byte[] encrypted = ProtectedData.Protect(jsonBytes, null, DataProtectionScope.CurrentUser);
```

Điều này giúp liên kết khóa mã hóa trực tiếp với tài khoản Windows hiện tại của người dùng, ngăn chặn hoàn toàn việc giải mã offline trên máy khác hoặc bởi các tiến trình độc hại khác.

2. Lỗi hỏng Path Traversal trong quản lý Backup (Medium Severity) [Trung Bình - Medium]

Vị trí: *JsonManager.cs (Dòng: 144-166, 168-213)*

Chi tiết: Trong hai phương thức SaveBackup(name) và LoadBackup(name, ...), ứng dụng nhận tham số chuỗi 'name' do người dùng hoặc giao diện cung cấp và ghép nối trực tiếp bằng Path.Combine(Paths.SavedBackups, name) mà không thực hiện bất kỳ thao tác chuẩn hóa hay kiểm tra tính hợp lệ nào.

Nguy cơ: Kẻ tấn công hoặc một lỗi nhập liệu có thể truyền vào chuỗi chứa các ký tự duyệt thư mục (như '../.../.../etc/passwd' hoặc '..\..\Windows\System32\cmd.exe') để ghi đè hoặc đọc trộm các tệp tin nằm ngoài thư mục chỉ định, đe dọa trực tiếp đến tính toàn vẹn của hệ thống.

Giải pháp đề xuất: Thực hiện kiểm tra chặt chẽ đường dẫn tuyệt đối trước khi thao tác file:

```
string fullPath = Path.GetFullPath(Path.Combine(Paths.SavedBackups, name));  
if (!fullPath.StartsWith(Paths.SavedBackups, StringComparison.OrdinalIgnoreCase))  
    throw new SecurityException("Truy cập ngoài thư mục backup bị cấm!");
```

3. Nguy cơ DLL Hijacking khi tải thư viện động (Medium Severity) [Trung Bình - Medium]

Vị trí: *Utility/RobloxDX12.cs (378, 413) & SwiftTunnel/NativeVpn.cs (P/Invokes) (Dòng: N/A)*

Chi tiết: Trong RobloxDX12, ứng dụng sử dụng LoadLibrary để nạp các DLL driver đồ họa như 'nvapi64.dll' và 'amd_ags_x64.dll' bằng tên tệp tin tương đối. Tương tự, NativeVpn định nghĩa DllImport cho 'swifttunnel_vpn.dll' mà không có sự kiểm tra vị trí tuyệt đối.

Nguy cơ: Nếu launcher JDKTrap được thực thi từ một thư mục không được bảo vệ (ví dụ: thư mục Downloads của người dùng), Windows sẽ ưu tiên tìm kiếm DLL trong thư mục khởi chạy trước. Kẻ tấn công có thể đặt một DLL giả mạo chứa mã độc trùng tên vào thư mục đó để ứng dụng nạp và chạy tự động khi khởi chạy.

Giải pháp đề xuất:

- Đối với thư viện hệ thống (như nvapi): Sử dụng LoadLibraryEx kết hợp với cờ hiệu LOAD_LIBRARY_SEARCH_SYSTEM32 để ép ứng dụng chỉ tìm kiếm trong thư mục C:\Windows\System32.
- Đối với thư viện đi kèm app (swifttunnel_vpn): Đảm bảo chỉ nạp từ thư mục cài đặt an toàn của chương trình (C:\Users\<User>\AppData\Local\JDKTrap\SwiftTunnel) và kiểm tra chữ ký điện tử trước khi nạp.

III. LỖI LOGIC HỆ THỐNG (LOGIC BUGS)

1. Hạ mức ưu tiên tiến trình Roblox xuống Below Normal [Nghiêm Trọng - High]

Vị trí: *Utility/RobloxMemoryHelper.cs (Dòng: 119)*

Chi tiết: Trong phương thức dọn RAM Roblox (CleanAllRobloxMemory), ứng dụng thực hiện lệnh:

```
SetPriorityClass(proc.Handle, BELOW_NORMAL_PRIORITY_CLASS);
```

Nguyên nhân là do lập trình viên nhầm lẫn rằng việc hạ ưu tiên sẽ giải phóng tài nguyên CPU cho các ứng dụng khác. Tuy nhiên, điều này trực tiếp phản tác dụng: Roblox sẽ bị hệ điều hành bóp hiệu năng CPU mỗi khi chạy ngầm hoặc khi hệ thống có tác vụ khác, gây tụt giảm FPS nghiêm trọng và gây giật lag (stuttering).

Giải pháp: Xóa bỏ hoàn toàn dòng lệnh hạ ưu tiên này. Thay vào đó, nếu muốn tối ưu hóa game, hãy nâng mức ưu tiên tiến trình Roblox lên ABOVE_NORMAL_PRIORITY_CLASS hoặc HIGH_PRIORITY_CLASS như đã làm ở các file khác.

2. Phản tác dụng hiệu năng từ EmptyWorkingSet [Trung Bình - Medium]

Vị trí: *Utility/RobloxMemoryHelper.cs (117) & RobloxDX12.cs (252) (Dòng: N/A)*

Chi tiết: Ứng dụng liên tục gọi `EmptyWorkingSet(proc.Handle)` để dọn RAM. API này ép buộc hệ điều hành đẩy toàn bộ trang bộ nhớ RAM vật lý đang hoạt động của Roblox vào bộ nhớ ảo Pagefile trên ổ cứng.

Hậu quả: Lượng RAM hiển thị trong Task Manager của Roblox sẽ giảm mạnh (tạo cảm giác tối ưu giả tạo). Nhưng ngay lập tức khi game cần truy cập lại dữ liệu, Windows sẽ phải đọc ngược dữ liệu từ ổ cứng (Page Faults) lên RAM, gây ra hiện tượng giật lag cực kỳ khó chịu khi đang chơi game.

Giải pháp: Loại bỏ hoàn toàn `EmptyWorkingSet`. Hãy để trình quản lý bộ nhớ ảo thông minh của Windows tự điều phối.

3. Sai sót logic trong việc kiểm tra Module DirectX 12 của Roblox [Trung Bình - Medium]

Vị trí: *Utility/RobloxDX12.cs (Dòng: 308-338)*

Chi tiết: Để kiểm tra Roblox có đang dùng DirectX 12 hay không, hàm `ProbeForDx12UsageSafe` gọi:

```
IntPtr h = GetModuleHandle("d3d12.dll");
```

Nguyên lý sai lầm: `GetModuleHandle` chỉ tìm kiếm trong không gian bộ nhớ của tiến trình hiện tại (`JDKTrap.exe`). Do đó, lệnh này chỉ cho biết JDKTrap có đang nạp `d3d12.dll` hay không, chứ hoàn toàn không có giá trị kiểm tra đối với tiến trình độc lập `RobloxPlayerBeta.exe`.

Giải pháp: Quét danh sách module thực tế của tiến trình Roblox mục tiêu:

```
bool hasDx12 = proc.Modules.Cast<ProcessModule>().Any(m => m.ModuleName.Equals("d3d12.dll", StringComparison.OrdinalIgnoreCase));
```

IV. ĐIỂM NGHẼN HIỆU SUẤT (PERFORMANCE BOTTLENECKS)

1. Cường bức gọi Garbage Collector định kỳ trong TrimTimer [Nghiêm Trọng - High]

Vị trí: *App.xaml.cs (hoặc Bloxstrap/App.xaml.cs) (Dòng: 410-442)*

Chi tiết: Trong `TrimTimer`, ứng dụng chạy một luồng lặp nền mỗi 5 giây để gọi `GC.Collect()` và nén LOH (Large Object Heap). Đây là một phản khuôn mẫu (antipattern) nghiêm trọng trong lập trình .NET.

Tác hại: Mỗi khi `GC.Collect` được gọi, toàn bộ luồng xử lý ứng dụng bị dừng lại (Stop-The-World). Việc nén LOH liên tục bắt CPU sao chép khối lượng lớn dữ liệu trong RAM, gây nghẽn `ThreadPool` và làm chậm máy vô ích.

Giải pháp: Xóa bỏ luồng `TrimTimer`. Bộ dọn rác .NET Core cực kỳ thông minh và tự động dọn dẹp hiệu quả.

2. Tái cấp phát liên tục tài nguyên đồ họa trong GPU Stress loop [Trung Bình - Medium]

Vị trí: `Utility/GPUOverclocker.cs` (Dòng: 600-659)

Chi tiết: Mỗi khi chạy vòng lặp stress GPU (`TryDispatchGpuWork`), ứng dụng lại tạo mới các đối tượng `DataStream`, `Buffer D3D11` và `UnorderedAccessView` trên GPU rồi giải phóng ngay lập tức.

Tác hại: Việc cấp/giải phóng liên tục hàng triệu byte tài nguyên VRAM trong vài giây gây phân mảnh bộ nhớ đồ họa, tăng driver overhead và làm nghẽn PCIe bus, làm suy giảm hiệu năng game Roblox đang chạy song song.

Giải pháp: Khởi tạo các tài nguyên này một lần duy nhất tại hàm `InitializeGpuResources`, lưu trữ dưới dạng các trường (fields) của class và tái sử dụng chúng. Chỉ gọi `Dispose` khi tắt tính năng.

V. BẢNG TỔNG HỢP KIẾN NGHỊ

Vấn đề phát hiện	Mức độ	Tệp tin ảnh hưởng	Khuyến nghị xử lý
XOR Obfuscation OAuth	Nghiêm trọng	SwiftTunnelAuthManager.cs	Mã hóa bằng DPAPI
Path Traversal Backup	Trung bình	JsonManager.cs	Sanitize & GetFullPath
DLL Hijacking LoadLibrary	Trung bình	RobloxDX12.cs / NativeVpn.cs	LOAD_LIBRARY_SEARCH_SYS32
Hạ ưu tiên Roblox	Nghiêm trọng	RobloxMemoryHelper.cs	Loại bỏ SetPriorityClass
Lạm dụng EmptyWorkingSet	Trung bình	MemoryHelper.cs / RobloxDX12.cs	Loại bỏ hoàn toàn API
DX12 GetModuleHandle Bug	Trung bình	RobloxDX12.cs	Quét Modules từ proc mục tiêu
GC.Collect 5s định kỳ	Nghiêm trọng	App.xaml.cs	Xóa bỏ luồng TrimTimer
GPU Allocations in loop	Trung bình	GPUOverclocker.cs	Khởi tạo 1 lần (Singleton)

Báo cáo được xuất tự động bởi trợ lý Antigravity AI.